Workshop 2

Maria Lucía Castillo García

Índice de entregables

1. **Imagen ubuntu-bash-lab** 2
2. **Sección de conclusiones** 2
3. **Evidencia de construcción y ejecución** 4

1. Imagen ubuntu-bash-lab

La imagen fue construida a partir de Ubuntu:latest mediante el comando de `docker build -t ubuntu-bash-lab .` y quedó registrada en el sistema local.

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-c
$ docker images

IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest      c4d56c24da4f       22.2MB         6.03MB    U
ejemplo-cmd:latest                   a30a7160aef4        157MB         41.6MB
ejemplo-combinado:latest             7f54bcab2e11        157MB         41.6MB
ejemplo-entrypoint:latest            603694ffee4c        157MB         41.6MB
japeto/pujgcc:latest                 9b3d7bbf4103        2.18GB         448MB    U
japeto/so-tools:v64                  7e990f123c8d          2GB         506MB    U
ubuntu-bash-lab:latest               c4ff932c0a29        162MB          43MB    U
ubuntu:latest                        2260313b31c8        160MB         45.3MB
```

→ Imagen `ubuntu-bash-lab:latest` con ID c4ff932c0a29, tamaño 162MB, construida sobre `ubuntu:latest`.

IMPORTANTE: El proceso completo de construcción se documenta en la sección de *Evidencia de Construcción y Ejecución* del presente texto.

2. Sección de conclusiones

A partir de la ejecución de los tres casos, se registraron los siguientes comportamientos:

OBSERVACIONES

	CASO A	CASO B	CASO C
Instrucciones	`CMD ["echo", "Hola desde CMD"]`	`ENTRYPOINT ["echo"]`	`ENTRYPOINT ["echo"] + CMD ["Mensaje por defecto"]`
Sin argumentos	Imprimió: `Hola desde CMD`	Imprimió una línea vacía (`echo` sin NADA que mostrar)	Imprimió: `Mensaje por defecto`
Con argumentos	Falló: `exec: "Mensaje reemplazado": executable file not found in $PATH`	Imprimió: `Hola` Y después: `Mensaje adicional`	Imprimió: `Mensaje personalizado`

¿Qué pasó con el argumento?	Reemplazó el `CMD` completo. Docker descartó el `echo` y trató de ejecutar `"Mensaje reemplazado"` como si fuera un programa instalado	Se sumó al `ENTRYPOINT` como parámetro. El programa siguió siendo `echo`, solo cambió lo que recibió	Reemplazó únicamente al `CMD`. el `ENTRYPOINT` quedó intacto y el argumentó entró como su nuevo parámetro

Con base a lo observado, se resumen las diferencias entre ambas instrucciones:

CMD vs ENTRYPOINT

	CMD	**ENTRYPOINT**
¿Qué define?	Lo que el contenedor hace si nadie le dice otra cosa, ya sea un comando o unos parámetros por defecto	El programa fijo que el contenedor ejecuta siempre pase lo que pase
Al pasar argumentos	Se reemplaza entero, no se le añade nada.	Se le suma como parámetros, el programa no cambia
¿Se puede sobrescribir sobre docker run?	Sí, basta con escribir algo al final del comando	No directamente, hay que usar la bandera `--entrypoint`
Analogía	Algo así como una sugerencia, o sea, "si no me pides nada, hago esto…"	Básicamente es como la identidad del contenedor, algo como: "yo soy esta herramienta, dime con qué trabajar".
¿Cuándo lo usaría?	Cuando quiero o necesito un contendor flexible, es decir, que el usuario pueda ejecutar lo que necesite (como el CMD ["bash"] de la Parte 1, pues se abre una terminal, pero permite correr otra cosa	Cuando quiero o necesito que el contenedor SEA una herramienta concreta y el usuario sólo decida sobre qué aplicarla

En primer lugar, una de las conclusiones sobre Docker que me impactan, es que éste no reconstruye todo desde cero cada vez. Va guardando capas y reutilizándolas cuando puede.

Ahora, sobre `CMD` y `ENTRYPOINT`, al principio pensaba que eran dos formas distintas de hacer lo mismo, y que daba igual usar cualquiera de los dos. Y claramente no lo eran, pero se pudo entender de mejor manera en los tres casos, en donde cada uno respondía a una pregunta

diferente. `ENTRYPOINT`, por ejemplo, responde a *qué es* el contenedor y `CMD` responde *qué hace por defecto*. Y precisamente por esto es que no compiten, sino que se complementan. El caso C fue el que me ayudó a entender más a profundidad, pues en éste el `echo` se mantuvo fijo mientras el mensaje cambiaba según lo que yo escribiera.

También me pareció muy curioso el error del Caso A, pues al pasarle "Mensaje reemplazado" esperaba que el contenedor imprimiera ese texto, y en cambio falló diciendo que no encontraba el ejecutable. Y comprendí que no era error como tal mío, sino que Docker muestra que un argumento no se le suma al `CMD`, sino que lo reemplaza entero. Entenderlo por las malas hizo que se me quedara mucho mejor que si simplemente hubiera funcionado.

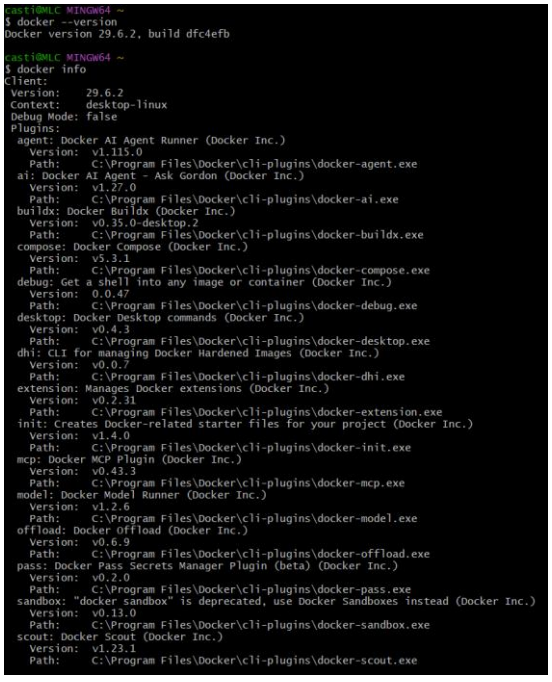
Por último, me parece que algo muy aplicable para la nube podría ser que si quiero que una imagen que funcione como una herramienta específica (procesar un archivo, validar datos, ejecutar un script de monitoreo) la combinación correcta es `ENTRYPOINT` para fijar el programa y `CMD` para los parámetros que el usuario pueda cambiar. Así quien la use no necesita saber nada de Docker, solo le pasa lo que quiere procesar y el contenedor se encarga del resto.

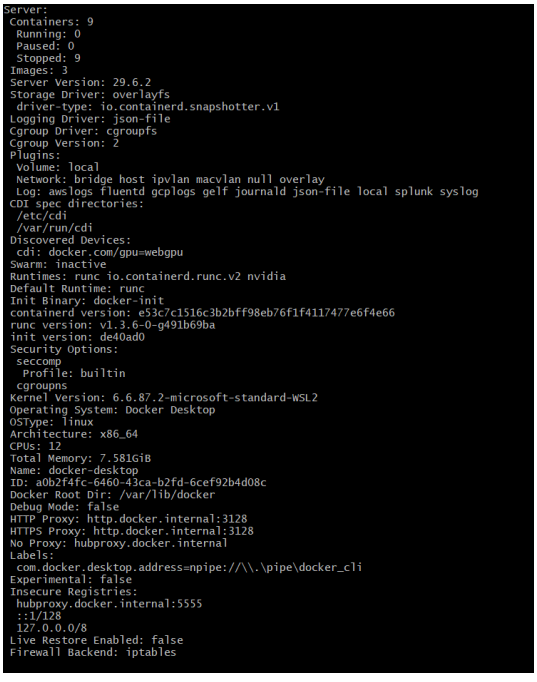
3. Evidencia de construcción y ejecución

Esta sección documenta cronológicamente todos los comandos ejecutados durante el taller, con sus respectivas salidas

Parte 1

1. Verificación de Docker





→ El daemon (proceso que administra Docker y se ejecuta en segundo plano (sin interacción directa del usuario)) responde: aparece la sección Server:

Esto confirma que el cliente Docker puede comunicarse correctamente con el servicio de Docker y que está funcionando.

2. **Creación de la carpeta y del contexto de construcción**

```
casti@MLC MINGW64 ~
$ cd ~

casti@MLC MINGW64 ~
$ mkdir -p workshop2-docker-clase-asincronica

casti@MLC MINGW64 ~
$ cd workshop2-docker-clase-asincronica/

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ pwd
/c/Users/casti/workshop2-docker-clase-asincronica
```

→ `pwd` confirma la ruta del taller.

3. **Descarga de la imagen base (docker pull Ubuntu):**

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker pull ubuntu:latest
latest: Pulling from library/ubuntu
f3db1cd94078: Pull complete
06e9d71331fb: Pull complete
f79afc4bbefe: Download complete
Digest: sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker images

IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest      c4d56c24da4f       22.2MB         6.03MB    U
japeto/pujgcc:latest                 9b3d7bbf4103       2.18GB          448MB    U
japeto/so-tools:v64                  7e990f123c8d          2GB          506MB    U
ubuntu:latest                        2260313b31c8        160MB         45.3MB
```

→ Se descarga la imagen base con el `docker pull ubuntu` y se muestra el mensaje Pull complete, indicando que la descarga finalizó correctamente, y la imagen aparece listada en `docker images`, confirmando que quedó almacenada localmente y disponible para crear contenedores.

4. Contenido del Dockerfile

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ cat Dockerfile
FROM ubuntu:latest

RUN apt-get update && apt-get install -y \
    bash \
    coreutils \
    grep \
    gawk \
    procps \
    nano \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY . /app

CMD ["bash"]
```

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ cat > saluuudis.sh << 'EOF'
> #!/bin/bash
> echo "Holaaaa, esto es un saludito desde un script dentro de la imageeen :)"
> echo "Usuario: $(whoami)"
> echo "Directorio actual: $(pwd)"
> EOF

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ chmod +x saluuudis.sh
```

→ Las cinco instrucciones: FROM, RUN, WORKDIR, COPY, CMD

5. Archivos del contexto

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ ls -la
total 30
drwxr-xr-x 1 casti 197609   0 Aug 27 16:52 ./
drwxr-xr-x 1 casti 197609   0 Aug 27 16:26 ../
-rw-r--r-- 1 casti 197609 183 Aug 27 16:46 Dockerfile
-rwxr-xr-x 1 casti 197609 148 Aug 27 16:52 saluuudis.sh*
```

6. Construcción de la imagen (docker Build)

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker build -t ubuntu-bash-lab .
[+] Building 26.2s (9/9) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 251B
 => [internal] load metadata for docker.io/library/ubuntu:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => CACHED [1/4] FROM docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
 => => resolve docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
 => [internal] load build context
 => => transferring context: 283B
 => [2/4] RUN apt-get update && apt-get install -y     bash     coreutils     grep     gawk     procps     nano     && rm -rf /var/lib/apt/lists/*
 => [3/4] WORKDIR /app
 => [4/4] COPY . /app
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:095b22c8e9f4e7a6700db90df1b6cbdcc6231c103734b5b24258a26191638a34
 => => exporting config sha256:17fbfef41db400c99da62f788ce5ec415dc5208f488ad85d67699a64a3d5f6d2
 => => exporting attestation manifest sha256:eb4d7af3f86c125c1301af3977e50127a5bbea47ed9c169e3b8af73fc25b3b13
 => => exporting manifest list sha256:c4ff932c0a298f19f36547982138266a27f890ebb712fba473a0123e45393b34
 => => naming to docker.io/library/ubuntu-bash-lab:latest
 => => unpacking to docker.io/library/ubuntu-bash-lab:latest

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker images

IMAGE                              ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest    c4d56c24da4f       22.2MB         6.03MB    U
japeto/pujgcc:latest               9b3d7bbf4103       2.18GB          448MB    U
japeto/so-tools:v64                7e990f123c8d          2GB          506MB    U
ubuntu-bash-lab:latest             c4ff932c0a29        162MB           43MB
ubuntu:latest                      2260313b31c8        160MB         45.3MB
```

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker ps -a
CONTAINER ID   IMAGE                              COMMAND                  CREATED        STATUS                      PORTS                    NAMES
521718d851b0   japeto/pujgcc:latest               "ls /home/"              6 months ago   Exited (0) 6 months ago                              hopeful_feynman
afb5eee01e76   japeto/pujgcc:latest               "ls /home"               6 months ago   Exited (0) 6 months ago                              nifty_nobel
d9be59b51ff5   japeto/pujgcc                      "bash"                   6 months ago   Exited (0) 6 months ago                              sad_golick
9c3c3c10f494   japeto/pujgcc                      "bash --help"            6 months ago   Exited (0) 6 months ago                              thirsty_raman
24c7c438be5b   japeto/so-tools:v64                "bash"                   6 months ago   Exited (255) 2 weeks ago                             pinito3
c88d9f5e2a5b   japeto/so-tools:v64                "bash"                   6 months ago   Exited (255) 2 weeks ago                             vigilant_albattani
df0fb71cea20   japeto/so-tools:v64                "bash"                   6 months ago   Exited (255) 2 weeks ago                             sweet_jones
f2380b79a3ea   japeto/so-tools:v64                "bash"                   6 months ago   Exited (255) 2 weeks ago                             exciting_villani
f84c80af6bca   docker/welcome-to-docker:latest    "/docker-entrypoint.…"   6 months ago   Exited (255) 6 months ago   0.0.0.0:8088->80/tcp     welcome-to-docker

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker run -it --name ubuntu-lab ubuntu-bash-lab
root@0d724e36cfd1:/app# pwd
/app
```

7. Imagen creada

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker run -it --name ubuntu-lab ubuntu-bash-lab
root@0d724e36cfd1:/app# pwd
/app
```

8. Ejecución del contenedor

```
root@0d724e36cfd1:/app# pwd
/app
root@0d724e36cfd1:/app# cd /app
root@0d724e36cfd1:/app# ls -la
total 16
drwxr-xr-x 1 root root 4096 Aug 28 10:18 .
drwxr-xr-x 1 root root 4096 Aug 28 10:26 ..
-rwxr-xr-x 1 root root  212 Aug 28 10:17 Dockerfile
-rwxr-xr-x 1 root root  148 Aug 27 21:52 saluuudis.sh
root@0d724e36cfd1:/app# ./saluuudis.sh
Holaaaa, esto es un saludito desde un script dentro de la imageeen :)
Usuario: root
Directorio actual: /app
root@0d724e36cfd1:/app# cat Dockerfile
FROM ubuntu:latest

RUN apt-get update && apt-get install -y \
    bash \
    coreutils \
    grep \
    gawk \
    procps \
    nano \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY . /app

CMD ["bash"]
root@0d724e36cfd1:/app# exit
exit
```

→ Ejecución del contenedor: pwd responde /app sin hacer cd → efecto de WORKDIR

→ Contenido copiado dentro del contenedor: ls -la en /app muestra los archivos → efecto de COPY

→ Ejecución del script dentro de la imagen: El script corre y reporta `Usuario: root`

9. Estados del contenedor

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker ps -a
CONTAINER ID   IMAGE                              COMMAND                  CREATED         STATUS                       PORTS                  NAMES
0d724e36cfd1   ubuntu-bash-lab                    "bash"                   5 minutes ago   Exited (0) About a minute ago                       ubuntu-lab
521718d851b0   japeto/pujgcc:latest               "ls /home/"              6 months ago    Exited (0) 6 months ago                             hopeful_feynman
afb5eee01e76   japeto/pujgcc:latest               "ls /home"               6 months ago    Exited (0) 6 months ago                             nifty_nobel
d9be59b51ff5   japeto/pujgcc                      "bash"                   6 months ago    Exited (0) 6 months ago                             sad_golick
9c3c3c10f494   japeto/pujgcc                      "bash --help"            6 months ago    Exited (0) 6 months ago                             thirsty_raman
24c7c438be5b   japeto/so-tools:v64                "bash"                   6 months ago    Exited (255) 2 weeks ago                            pinito3
c88d9f5e2a5b   japeto/so-tools:v64                "bash"                   6 months ago    Exited (255) 2 weeks ago                            vigilant_albattani
df0fb71cea20   japeto/so-tools:v64                "bash"                   6 months ago    Exited (255) 2 weeks ago                            sweet_jones
f2380b79a3ea   japeto/so-tools:v64                "bash"                   6 months ago    Exited (255) 2 weeks ago                            exciting_villani
f84c80af6bca   docker/welcome-to-docker:latest    "/docker-entrypoint.…"   6 months ago    Exited (255) 6 months ago    0.0.0.0:8088->80/tcp   welcome-to-docker
```

→ Vacío vs. `Exited` → el contenedor existe, aunque no corra.

10. Registro de la sesión:

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker logs ubuntu-lab
root@0d724e36cfd1:/app# pwd
/app
root@0d724e36cfd1:/app# cd \app
bash: cd: app: No such file or directory
root@0d724e36cfd1:/app# pwd
/app
root@0d724e36cfd1:/app# /
.dockerenv  app/        boot/       etc/        lib/        media/      opt/        root/       sbin/       sys/        usr/
.rock/      bin/        dev/        home/       lib64/      mnt/        proc/       run/        srv/        tmp/        var/
root@0d724e36cfd1:/app# cd /app
root@0d724e36cfd1:/app# exit
exit
root@0d724e36cfd1:/app# pwd
/app
root@0d724e36cfd1:/app# cd /app
root@0d724e36cfd1:/app# ls -la
total 16
drwxr-xr-x 1 root root 4096 Aug 28 10:18 .
drwxr-xr-x 1 root root 4096 Aug 28 10:26 ..
-rwxr-xr-x 1 root root  212 Aug 28 10:17 Dockerfile
-rwxr-xr-x 1 root root  148 Aug 27 21:52 saluuudis.sh
root@0d724e36cfd1:/app# ./saluuudis.sh
Holaaaa, esto es un saludito desde un script dentro de la imageeen :)
Usuario: root
Directorio actual: /app
root@0d724e36cfd1:/app# cat Dockerfile
FROM ubuntu:latest

RUN apt-get update && apt-get install -y \
    bash \
    coreutils \
    grep \
    gawk \
    procps \
    nano \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY . /app

CMD ["bash"]
root@0d724e36cfd1:/app# exit
exit
```

→ Recupera la salida de una sesión ya cerrada

11. Administración

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker start ubuntu-lab
ubuntu-lab

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker exec -it ubuntu-lab whoami
root

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug ubuntu-lab
    Learn more at https://docs.docker.com/go/debug-cli/

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ docker stop ubuntu-lab
ubuntu-lab
```

→ `exec` responde `root` en un contenedor ya en ejecución

==========

Parte 2

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ cd ~/workshop2-docker-clase-asincronica

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ mkdir -p caso-a caso-b caso-c
```

A: SOLO `CMD`

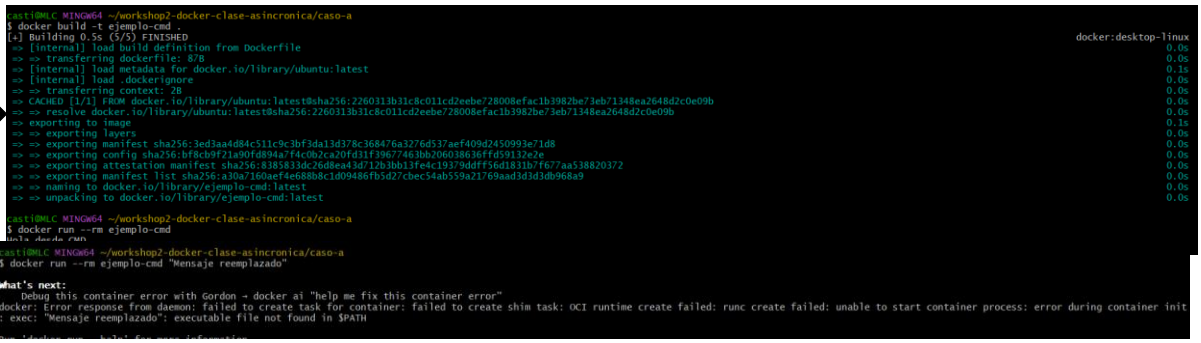
A.1. Dockerfile del caso A

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica
$ cd caso-a/

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-a
$ cat > Dockerfile << 'EOF'
> FROM ubuntu:latest
> CMD ["echo", "Hola desde CMD"]
> EOF

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-a
$ cat Dockerfile
FROM ubuntu:latest
CMD ["echo", "Hola desde CMD"]
```

A.2. Ejecución sin argumentos



→ Aparece el executable file not found, y esto es porque el argumento reemplazó al CMD completo

---

B: Solo `ENTRYPOINT`

B.1. Dockerfile del caso B

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-a
$ cd ../caso-b

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-b
$ cat > Dockerfile << 'EOF'
> FROM ubuntu:latest
> ENTRYPOINT ["echo"]
> EOF

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-b
$ docker build -t ejemplo-entrypoint .
[+] Building 0.5s (5/5) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 76B
 => [internal] load metadata for docker.io/library/ubuntu:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => CACHED [1/1] FROM docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
 => => resolve docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:e98a316fa79fd45fd55f4a89c65093ae3375bbf5403f90fdd0fa105143d67287
 => => exporting config sha256:73228b9a057d67d302bb2bf72144651db07dc5d1bacb047df8d9b8b9c59b636d
 => => exporting attestation manifest sha256:bd71cac79b03a1dcf815e9ca29ab7e3d4344f208411ed16b6c52f4093b909a4b
 => => exporting manifest list sha256:603694ffee4cbf0f6b14bee8f0d3b03142cea6f8cd7eda179e8fa616b6291e37
 => => naming to docker.io/library/ejemplo-entrypoint:latest
 => => unpacking to docker.io/library/ejemplo-entrypoint:latest
```

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-b
$ docker images

IMAGE                              ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest    c4d56c24da4f       22.2MB         6.03MB    U
ejemplo-cmd:latest                 a30a7160aef4        157MB         41.6MB
ejemplo-entrypoint:latest          603694ffee4c        157MB         41.6MB
japeto/pujgcc:latest               9b3d7bbf4103       2.18GB          448MB    U
japeto/so-tools:v64                7e990f123c8d         2GB          506MB    U
ubuntu-bash-lab:latest             c4ff932c0a29        162MB           43MB    U
ubuntu:latest                      2260313b31c8        160MB         45.3MB
```

B.2. Ejecución con distintos argumentos

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-b
$ docker run --rm ejemplo-entrypoint "Hola"
Hola

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-b
$ docker run --rm ejemplo-entrypoint "Mensaje adicional"
Mensaje adicional

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-b
$ docker run --rm ejemplo-entrypoint
```

→ Siempre se ejecuta echo, el argumento se le suma

C: `ENTRYPOINT` y `CMD`

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-b
$ cd ../caso-c/
```

C.1. Dockerfile del caso C

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-c
$ cat > Dockerfile << 'EOF'
> FROM ubuntu:latest
> ENTRYPOINT ["echo"]
> CMD ["Mensaje por defecto"]
> EOF

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-c
$ cat Dockerfile
FROM ubuntu:latest
ENTRYPOINT ["echo"]
CMD ["Mensaje por defecto"]

casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-c
$ docker build -t ejemplo-combinado .
[+] Building 0.4s (5/5) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 104B
 => [internal] load metadata for docker.io/library/ubuntu:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => CACHED [1/1] FROM docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
 => => resolve docker.io/library/ubuntu:latest@sha256:2260313b31c8c011cd2eebe728008efac1b3982be73eb71348ea2648d2c0e09b
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:d4f9e5e34f52fae5eefa05060fd9153d58476371da2d3ed33d5ad0a6f807acfd
 => => exporting config sha256:b139486132307837949646b89890488e6ef915bedf12ece9df6e5c36bf8629a3
 => => exporting attestation manifest sha256:a1b672a849f97e710939ada54fa10affcbfcb5480606e624137fdf23dacbc70c
 => => exporting manifest list sha256:6dc4e60e4e66b0417d87ea67045c35cb02cc75dea0b02de095468594dba7b7eb
 => => naming to docker.io/library/ejemplo-combinado:latest
 => => unpacking to docker.io/library/ejemplo-combinado:latest
```

C.2. Sin argumentos (valor por defecto)

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-c
$ docker run --rm ejemplo-combinado
Mensaje por defecto
```

→ Imprime “Mensaje por defecto”

C.3. Con argumento (parámetro)

```
casti@MLC MINGW64 ~/workshop2-docker-clase-asincronica/caso-c
$ docker run --rm ejemplo-combinado "Mensaje personalizado"
Mensaje personalizado
```

→ El programa no cambia, solo lo que recibe

==========

Preguntas de análisis

1) ¿Cuál instrucción se comporta como valor por defecto?

- Sería `CMD` porque se ejecuta únicamente cuando el usuario no escribe nada adicional al hacer `docker run`. Por ejemplo, como se vio en el caso C, que cuando se ejecutó la imagen sin argumento apareció “Mensaje por defecto”; pero al escribir “Mensaje personalizado” en `docker run`, Docker usó ese nuevo valor en lugar del que estaba en `CMD`. Eso muestra que `CMD` sirve para definir qué hacer o qué usar cuando el usuario no especifica otra opción.

2) ¿Cuál instrucción define el programa principal del contenedor?

- `ENTRYPOINT`, porque determina qué programa se ejecutará siempre. En el caso B, sin importar si yo escribía “Hola” o “Mensaje adicional”, Docker seguía ejecutando echo; donde lo único que cambiaba era el texto que recibía. Por eso el `ENTRYPOINT` fija el ejecutable principal.

3) ¿Qué sucede con `CMD` cuando se pasa un comando al final de `docker run`?

- Docker reemplaza completamente el contenido de `CMD`. Como se pudo evidenciar en el caso A, la imagen estaba configurada para ejecutar echo “Hola desde `CMD`”, pero al pasar “Mensaje reemplazado”, Docker ignoró lo definido en `CMD` e intentó usar ese nuevo valor en su lugar.

4) ¿Por qué `ENTRYPOINT` es útil cuando queremos que el contenedor se comporte como una herramienta concreta?
- Porque básicamente hace que el contenedor SEA esa herramienta. Por ejemplo, con el `ENTRYPOINT ["echo"]`, mi imagen dejó de ser un "Ubuntu" y pasó a ser un "`echo`", en donde uno la usa igual que usaría el comando, pasándole lo que quiere imprimir. Entonces, si en lugar de echo fuera un programa que comprime archivos o valida datos, el usuario no tendría que saber nada de Docker, pues bastaría con que solo le pase el archivo y ya.

5) ¿Qué combinación utilizaría para crear un script que tenga un ejecutable fijo, pero permita modificar parámetros?
- Usaría `ENTRYPOINT` junto a `CMD`. `ENTRYPOINT` deja fijo el programa que se ejecutará y `CMD` proporciona los parámetros iniciales. Como se mencionó en las preguntas anteriores, `ENTRYPOINT` deja fijo el programa que se ejecutará y `CMD` proporciona los parámetros iniciales o por defecto (que el usuario puede cambiar sin tocar la imagen).